

第2章 运算符与表达式

建议学时:3-5 小时 | 难度:★★★★☆ | 读者背景:已掌握 C 语言

🎯 本章学习目标

- 掌握 JS 算术运算与 C 的差异:双精度溢出、除法语义、负数取模、** 指数
- 理解位运算的 32 位有符号整数限制与无符号右移 >>>
- 讲透逻辑运算符返回 **操作数本身** 而非布尔值的机制,含 ?? 空值合并
- 彻底掌握 === 与 ==:真值表、隐式转换的著名坑(0=='0'、null>=0、[]==![])
- 熟练使用解构赋值、展开运算符、可选链 ?.
- 掌握运算符优先级表,理解求值顺序与 C 的差异
- 理解"C 中怎么做→JS 中怎么做":C 的表达式是纯数值运算,JS 的表达式经常是"值选择"

💡 从 C 到 JavaScript:本章主题如何迁移

运算符是两种语言最"像"的部分——+ - * / %、位运算、逻辑与比较,符号几乎完全一样,连优先级表都大同小异。但你若照搬 C 的心智模型,第 2 天就会在生产代码里踩坑。

第一个差异是 **除法**:C 里 5/2 是 2(截断),JS 里 5/2 是 2.5。第二个差异是 **逻辑运算符的返回值**:C 的 a && b 永远得到 0 或 1,JS 的 a && b 返回 a 或 b 的 **原值**。第三个差异是相等比较:JS 的 == 会做隐式类型转换,产生 0 == '' 为真这样的"鬼故事",因此生产规范几乎一律使用 ===。

第四个差异是 **位运算只对 32 位整数有效**——0xFFFFFFFF 参与位运算会得到 -1,与 C 的 64 位直觉完全不同。

第五个差异是 JS 补了三个 C 没有的实用运算符:指数 **、可选链 ?.、空值合并 ??,它们让"安全取值"变得极其简洁。

读完本章,你应该能对任何一段 JS 表达式做两件事:① 判断它输出什么(值);② 说出它为什么(规则)。这两件事是对一切代码评审的基础。

📦 前置知识自查

- C 的算术/关系/逻辑/位/赋值运算符及优先级
- C 的短路求值:&& 与 || 只算必要的一边
- C 的整数除法截断与负数取模结果(C99 规定向零截断)
- C 的复合赋值 +=、-=、*=(注意 *= 与 ++ 的区别)
- C 的强制转换 (int) 与隐式整型提升

🚀 本章学习路线

1. **第一阶段(算术与位运算)**:读 §2.1–§2.3,重点建立"number 是浮点"的运算直觉
2. **第二阶段(逻辑与比较)**:读 §2.4–§2.5,亲手在 REPL 验证每个真值表条目,这是本章价值核心
3. **第三阶段(赋值与优先级)**:读 §2.6–§2.8,做优先级小测验
4. **第四阶段(实验)**:完成章末"表达式求值器"
5. **验收标准**:能不看资料说出 `0==''`、`null>=0`、`[]==![]` 三个表达式的值及原因

本章知识导图

- 第2章 运算符与表达式
 - §2.1 算术运算:除法语义、溢出、取模、指数 `**`
 - §2.2 自增自减:前置/后置
 - §2.3 位运算:32 位限制、`>>>`、标志位场景
 - §2.4 逻辑与短路:返回操作数、`??` 空值合并
 - §2.5 比较:`===` 与 `==` 真值表、NaN、Object.is
 - §2.6 赋值、解构与展开
 - §2.7 三目运算符与表达式风格
 - §2.8 优先级与求值顺序
 - §2.9 特色运算符:可选链 `?.`、`in`、`instanceof`

§2.1 算术运算:五种与 C 的本质差异

运算	C 的行为	JS 的行为
<code>5 / 2</code>	2(int 除法截断)	2.5(永远是浮点除法!)
<code>5 % 2</code>	1	1(与 C 一致,向零取整)
<code>-5 % 2</code>	-1(C99 向零截断)	-1(符号随被除数)
溢出	有符号 UB,无符号回绕	变成 Infinity,永不崩溃
除 0	运行时错误/UB	<code>1/0 = Infinity</code> , <code>0/0 = NaN</code>
指数	用 <code>pow()</code> 函数	<code>**</code> 运算符与 <code>Math.pow</code>

```

console.log(5 / 2);           // 2.5 ← 没有整数除法!想取整用 Math.floor
console.log(Math.floor(5 / 2)); // 2
console.log(5 % 2);           // 1
console.log(-5 % 2);           // -1(结果符号跟随被除数,和 C 相同)
console.log(5 % -2);           // 1
console.log(2 ** 10);          // 1024,指数运算符(ES2016)
console.log(2 ** 0.5);         // 1.4142135623730951,支持浮点指数
console.log(1 / 0);            // Infinity
console.log(-1 / 0);           // -Infinity
console.log(0 / 0);            // NaN

// number 的精度:超过安全整数就失真
console.log(0.1 + 0.2);        // 0.30000000000000004(浮点误差,C 也一样)
console.log(Number.MAX_VALUE * 2); // Infinity,真正的溢出

```

2.1.1 取整三兄弟:floor / ceil / trunc

C 的 (int)x 是向零截断;JS 没有 (int),对应的是 `Math.trunc`。负数场景必须选对:

```

console.log(Math.floor(-1.5)); // -2 ← 向下取整(向负无穷)
console.log(Math.ceil(-1.5));  // -1 ← 向上取整
console.log(Math.trunc(-1.5)); // -1 ← 向零截断,等价于 C 的 (int)(-1.5)
console.log(Math.round(-1.5)); // -1 ← 四舍五入,注意 -1.5 向 -1 走!
console.log(Math.round(2.5));  // 3(正数 0.5 进位,负数有坑)

```

⚠ 常见误区 2.1:用 `parseInt` 代替 `Math.floor`

`parseInt(-0.5)` 得 -0(`parseInt` 只处理字符串/数字的整数部分,先转字符串再解析); `Math.floor(-0.5)` 得 -1。两者对负数的语义完全不同。生产规范:数值取整只用 `Math.floor/trunc/round`,`parseInt` 只用于"字符串转整数"。

§2.2 自增自减:与 C 完全一致

```

let i = 5;
console.log(i++); // 5(后置:先用后加)
console.log(i);   // 6
console.log(++i); // 7(前置:先加后用)
console.log(i);   // 7

// 注意:字符串参与算术运算会隐式转数字(第2.5节详细讲)
let s = '3';
console.log(s + 1); // '31' ← + 碰到字符串就变拼接!
console.log(s - 1); // 2    ← - * / 则会转成数字
console.log(+s);    // 3    ← 一元 + 是"转数字"的惯用法

```

⚠ 常见误区 2.2:+ 是"又加又拼"的变色龙

C 的 `+` 只做加法;JS 的 `+` 只要任一边是字符串,就变成字符串拼接: `1 + 2 + '3'` 是 '33'(先 `1+2=3` 再拼 '3'),而 `'1' + 2 + 3` 是 '123'。生产铁律:拼接用模板字符串,加法要保证两边都是 `number`,否则用 `Number()` 显式转换。

§2.3 位运算:32 位有符号整数的世界

JS 的所有位运算符(& | ^ ~ << >> >>>)都会把操作数先转成 32 位有符号整数再运算,结果也按 32 位有符号解释。这意味着:

```
console.log(1 << 31);           // -2147483648 ← 溢出成负数了!
console.log((1 << 31) >>> 0);    // 2147483648 ← 无符号右移修正
console.log(0xFFFFFFFF);        // 4294967295(字面量是普通 number)
console.log(0xFFFFFFFF | 0);     // -1 ← 位运算后变成 32 位有符号!
console.log(~0);                 // -1(取反,和 C 一样)
console.log(7 & 3);              // 3(按位与)
console.log(7 | 8);              // 15(按位或)
console.log(7 ^ 3);              // 4(按位异或)
console.log(8 >> 1);             // 4(有符号右移,补符号位)
console.log(-8 >> 1);            // -4
console.log(-8 >>> 1);           // 2147483644(无符号右移,高位补 0)
```

★ 重点结论:JS 没有 >>> 之外,位运算世界 = 32 位

参与位运算的 number 先做 `ToInt32`:超过 32 位范围的数值会被截断。所以位运算不能用于大整数 (ID 字段、时间戳),那是 `BigInt` 的领域(`10n & 3n` 是合法的 `BigInt` 位运算)。32 位限制的最大实用价值:位运算很快,常用于标志位组合与状态压缩。

2.3.1 生产场景:权限标志位(与 C 的 enum 位标志对照)

```
// C: enum Perm { READ=1, WRITE=2, EXEC=4 }; int p = READ|WRITE;
const PERM = Object.freeze({
  READ: 1, // 0b001
  WRITE: 2, // 0b010
  EXEC: 4, // 0b100
});
let perm = PERM.READ | PERM.WRITE; // 3 = 0b011,组合权限
console.log(perm & PERM.READ);     // 1,非零即有读权限
console.log(perm & PERM.EXEC);     // 0,无执行权限
perm |= PERM.EXEC;                 // 7 = 0b111,追加权限
console.log(perm);                 // 7
perm &= ~PERM.WRITE;                // 5 = 0b101,撤销写权限
console.log(perm);                 // 5
console.log((perm & PERM.WRITE) !== 0); // false,确认写权限已撤销
```

⚠ 常见误区 2.3:用 && 判断位标志

`perm && PERM.READ` 逻辑错误——&& 判断的是真值不是位!判断位必须用 `(perm & PERM.READ) !== 0`,不要用 `perm && x` 这种“巧写”,它会把 0 以外的任何值当“有”。位运算与逻辑运算的混用在代码评审中很常见,务必分清。

§2.4 逻辑与短路:返回的是操作数,不是布尔

这是 JS 与 C 最大的运算符差异: `&&` 与 `||` 返回两个操作数之一,不强制转布尔。C 里 `a && b` 是 0/1;JS 里:

```
// || :返回第一个"真值"操作数;都假则返回最后一个
console.log(0 || '默认值'); // '默认值' ← 经典默认值写法
console.log('' || 'fallback'); // 'fallback'(空串是假值)
console.log(null || 42); // 42
console.log(undefined || 1); // 1
console.log(3 || 99); // 3 ← 左边真就直接返回,右边不评估
console.log(0 || false); // false ← 都假,返回最后一个

// && :返回第一个"假值"操作数;都真则返回最后一个
console.log(1 && 2); // 2 ← 都真返回最后一个
console.log(0 && 2); // 0
console.log('' && 'x'); // ''
console.log(null && 'x'); // null
console.log(true && 'ok'); // 'ok'

// 短路:右边不执行
let called = 0;
false && (called = 1); // 右边不执行
console.log(called); // 0
true || (called = 2); // 右边不执行
console.log(called); // 0

// !! 双取反:转成真正的布尔
console.log(!!'abc'); // true
console.log(!!0); // false
```

2.4.1 假值全集:只有 6 个

JS 的真值转换只有 6 个假值: false、0、-0、""(空串)、null、undefined、NaN。其余一切都是真值,包括 '0'、'false'、[]、{}。这与 C 的"0 假非 0 真"完全不同——C 里任何非零整数都是真,JS 里 '0' 是真、[] 也是真。

```
const falsy = [false, 0, -0, '', null, undefined, NaN];
for (const v of falsy) {
  console.log(String(v).padEnd(9), '->', Boolean(v)); // 全部 false
}
console.log(Boolean('0')); // true ← 字符串 '0' 不是假值!
console.log(Boolean([])); // true
console.log(Boolean({})); // true
console.log(Boolean('false')); // true
console.log(Boolean(Infinity)); // true
```

2.4.2 空值合并 ?? :只有 null/undefined 才触发

?? 与 || 的区别: || 对所有假值生效,?? 只对 null/undefined 生效。处理"数值 0 也是合法配置"的场景必须用 ??:

```
const retries = 0;
console.log(retries || 3);    // 3 ← 坑!0 被 || 吞掉了
console.log(retries ?? 3);    // 0 ← 正确,0 是合法值

const name = '';
console.log(name || '匿名');  // '匿名'
console.log(name ?? '匿名');  // '' ← 空串也是合法值

// 注意:?? 不能与 && || 混用不括号
// console.log(1 ?? 2 || 3); // ❌ SyntaxError,必须 (1 ?? 2) || 3
```

§2.5 比较:=== 与 == 的完整真相

2.5.1 严格相等 ===:生产唯一选择

=== 不做类型转换,类型不同直接不相等。与 C 的 **==** 直觉一致,只是把"类型必须匹配"提到了第一位。判空、判枚举、判结果一律用 **===**。两个坑:NaN 永远不等于任何值(含自身);对象比较的是引用不是内容(和 C 的结构体按位比较不同)。

```
console.log(1 === 1);          // true
console.log(1 === '1');        // false ← 类型不同
console.log('1' === '1');      // true
console.log(NaN === NaN);       // false ← 唯一的不自等值
console.log(0 === -0);          // true(=== 不区分正负零)
console.log({} === {});        // false ← 不同对象,引用不同
const a = { n: 1 };
const b = a;
console.log(a === b);           // true ← 同一引用
console.log(null === undefined); // false
```

2.5.2 宽松相等 ==:你真要用它,先把这张表背下来

== 会先做类型转换再比较。它存在完全是因为历史原因,生产代码评审几乎一律禁止使用。但读老代码时必须能看懂,所以列出核心规则与著名例子:

```
// 数字 vs 字符串:字符串转数字后比较
console.log(0 == '0');           // true ← 著名坑
console.log(1 == '1');           // true
console.log('' == 0);            // true ← 空串转 0

// 布尔参与:先转数字(true=1, false=0)
console.log(true == 1);          // true
console.log(false == 0);         // true
console.log(true == 2);          // false
console.log('true' == true);    // false ← 字符串 'true' 不是数字,转 NaN

// null 与 undefined:只与彼此相等
console.log(null == undefined); // true
console.log(null == 0);         // false ← 著名坑,null 不转 0!
console.log(undefined == 0);    // false

// 对象与原始值:对象先转原始值(toString/valueOf)
console.log([] == '');          // true ← [] 转成 ''
console.log([] == 0);           // true
console.log([1] == 1);          // true
console.log([1, 2] == '1,2');   // true
console.log([] == ![]);         // true ← 最著名的鬼故事!
console.log('' == false);       // true

// 关系比较的坑:null >= 0 为 true 而 null > 0 为 false
console.log(null > 0);           // false(null 转 0,0 > 0 为假)
console.log(null == 0);         // false(== 特殊规则,不转换)
console.log(null >= 0);         // true ← 无解之谜!>= 里 null 转 0
console.log(undefined >= 0);     // false(undefined 转 NaN)
console.log(NaN >= 0);          // false(NaN 与任何比较都是 false)
```

★ 必须记住的结论

① 全部用 ===,除非你有意做 `x == null` 的判空(它同时匹配 null 与 undefined,这是 == 唯一的生产用法);② 关系比较(> < >= <=)只对 number 使用,字符串比较是字典序;③ NaN 永远不比;④ 判相等永远不用 == 处理对象。

2.5.3 Object.is:比 === 更严格

```
console.log(Object.is(NaN, NaN)); // true ← 唯一能判 NaN 相等的方法(其实 Number.isNaN 也行)
console.log(Object.is(0, -0));    // false ← 区分正负零
console.log(Object.is(1, '1'));   // false
console.log(Object.is({}, {}));   // false
```

§2.6 赋值、解构与展开

2.6.1 复合赋值

```
let x = 10;
x += 5;    // 15
x -= 3;    // 12
x *= 2;    // 24
x /= 4;    // 6
x %= 4;    // 2
x **= 3;   // 8,指数复合赋值
x <<= 1;   // 16,位移复合赋值
x >>= 2;   // 4
x &= 3;    // 0(按位与)
x |= 8;    // 8(按位或)
console.log(x);
```

2.6.2 解构赋值:C 没有对应物

```
// 数组解构:一次性取出多个值
const point = [3, 7];
const [px, py] = point;
console.log(px, py);           // 3 7

// 交换变量(不需要临时变量,没有 C 的 tmp 写法)
let a2 = 1, b2 = 2;
[a2, b2] = [b2, a2];
console.log(a2, b2);           // 2 1

// 对象解构:按属性名取值
const user = { id: 1, name: '韩梅梅', age: 17 };
const { name, age } = user;
console.log(name, age);        // 韩梅梅 17

// 重命名 + 默认值
const { name: userName = '匿名', city = '北京' } = user;
console.log(userName, city);    // 韩梅梅 北京

// 剩余元素 ...
const [first, ...rest] = [10, 20, 30, 40];
console.log(first, rest);       // 10 [ 20, 30, 40 ]
```


2.6.3 展开运算符 ...

```
// 数组展开:合并数组(替代 C 的 memcpy 手写循环)
const arr1 = [1, 2];
const arr2 = [3, 4];
const merged = [...arr1, ...arr2];
console.log(merged);           // [ 1, 2, 3, 4 ]

// 数组拷贝(浅拷贝)
const copy = [...merged];
console.log(copy === merged);  // false, 是新的数组

// 对象展开:合并/覆盖属性(浅拷贝)
const base = { host: 'localhost', port: 3000 };
const cfg = { ...base, port: 8080, debug: true };
console.log(cfg);              // { host: 'localhost', port: 8080, debug: true }

// 函数调用展开
console.log(Math.max(...[3, 9, 5])); // 9, 替代 apply
```

⚠ 常见误区 2.4:展开是浅拷贝

`{...obj}` 只拷贝一层,嵌套对象仍然是共享引用。修改拷贝后的嵌套对象会同时影响原对象——与 C 的 `memcpy` 结构体完全不同的语义。深拷贝的正确姿势见第14章(`structuredClone`)。

§2.7 三目运算符 ?:

C 的三目在 JS 中完全保留,且因为表达式风格流行,使用频率远高于 C。生产风格建议:单层三目可读性很好,嵌套三目禁止(可读性灾难,改用函数或 if)。

```
const score = 85;
const level = score >= 60 ? '及格' : '不及格';
console.log(level);           // 及格

// 三目是"表达式",可以返回值参与后续运算(与 C 一致)
const price = 100;
const pay = price > 50 ? price * 0.9 : price;
console.log(pay);             // 90

// ❌ 禁止嵌套三目:score>90?'A':score>80?'B': 'C'
// 应改写为:
function levelOf(s) {
  if (s > 90) return 'A';
  if (s > 80) return 'B';
  return 'C';
}
console.log(levelOf(85));      // B
```

§2.8 优先级与求值顺序

JS 的优先级表与 C 高度相似,只需记住几个差异点。下面按优先级从高到低列出常用运算符(完整表见 MDN):

优先级	运算符	与 C 的差异
最高	() 分组、[] 下标、. 属性、?. 可选链、new	新增 ?.
2	++ -- 一元 - + ! ~ typeof	一元 + 是转数字
3	** 指数	C 没有;注意它优先于负号
4	* / %	同 C
5	+ -	+ 可能是拼接!
6	<< >> >>>	新增 >>>
7	< <= > >= in instanceof	新增 in/instanceof
8	== != === !==	新增 === !==
9	&	同 C
10	^	同 C
11		同 C
12	&&	返回操作数
13		返回操作数
14	??	C 没有,不能与 12/13 混用不括号
15	? :	同 C
16	= += -= ...	同 C
最低	, 逗号	同 C

★ 求值顺序:C 的 UB,JS 的规定

C 的 `f() + g()` 谁先调用是未定义行为;JS 规定 从左到右。另外 JS 没有序列点问题, `i = i++` 的后果是确定的(虽然仍然是无意义代码)。生产建议:与 C 相同,一行表达式不要写两个副作用。

```
// 优先级实战
console.log(2 ** 3 ** 2);    // 512 ← ** 是右结合!3**2=9,2**9=512
console.log(-2 ** 2);        // SyntaxError ← 必须写 -(2**2)
console.log(-(2 ** 2));      // -4
console.log(3 + 4 * 2);       // 11,乘法先
console.log((3 + 4) * 2);     // 14,括号
console.log(1 < 2 == true);   // true:1<2 是 true,true==true
console.log(0 & 1 || 2);      // 2:位运算优先于逻辑运算
// 口诀:括号永远是最可靠的自文档—不要靠背表
```

§2.9 特色运算符:可选链 ?. 与 in/instanceof

2.9.1 可选链 ?.:C 没有的安全访问

```
const resp = { data: { user: { name: '李雷' } } };
```

// C 写法:每层都要判 NULL(或调用方保证非空)
// JS 老写法:
const n1 = resp && resp.data && resp.data.user && resp.data.user.name;
// JS 现代写法:
const n2 = resp?.data?.user?.name;
console.log(n2); // 李雷
console.log(resp?.data?.list?.[0]?.id); // undefined,中间断链不报错
console.log(user?.name ?? '匿名'); // 与 ?? 配合是生产标配

// 注意:?. 只短路"属性访问",不短路后面的方法调用
// resp.missing.hello() ← 报错;resp.missing?.hello() ← undefined

2.9.2 in 与 instanceof

```
const obj = { a: 1, b: undefined };  
console.log('a' in obj); // true  
console.log('b' in obj); // true ← 属性存在但值是 undefined  
console.log('c' in obj); // false  
console.log(obj.c === undefined); // true,in 能区分"属性不存在"与"属性为 undefined"
```



```
console.log([] instanceof Array); // true  
console.log({} instanceof Object); // true  
console.log(/re/ instanceof RegExp); // true  
// 跨 realm(iframe/worker)时 instanceof 会失效,生产用 Array.isArray  
console.log(Array.isArray([])); // true
```

工程建议 2.1:表达式也要自文档化

三条规定:① 逻辑与位运算混写必须加括号;② 比较运算的操作数先显式转 number,杜绝隐式转换混入表达式;③ 复杂表达式拆成带名字的局部变量(`const isAdmin = user.role === 'admin'`),让代码评审者一眼看懂意图。

工程建议 2.2:判空统一风格

团队里统一三种判空:期望 null/undefined 都没有时用 `x == null`;期望"有实际值"时用 `x !== null && x !== undefined` 或 `x != null`;只关心真值(如布尔开关)时用 `if (x)`。ESLint 配置 `eqeqeq` 规则可以强制 `===`,把 `==` 限制在判空场景。

本章速查表

主题	速查要点
除法	/ 永远是浮点除法:5/2=2.5;整数除法用 Math.floor(a/b)
取模	结果符号跟随被除数,与 C99 一致;-5%2=-1
溢出	number 溢出得 Infinity,不崩溃;精确整数限 $\pm 2^{53}-1$
除零	1/0=Infinity,0/0=NaN,不报错,需自行检查
指数	2 ** 10 = 1024;右结合;-2**2 需写 -(2**2)
位运算	先转 32 位有符号整数:0xFFFFFFFF 0 = -1;大整数用 BigInt
>>>	无符号右移,高位补 0;C 没有
假值	仅 6 个:false 0 -0 '' null undefined NaN
&& /	返回操作数原值; 取第一个真值, 都假取最后;短路
??	只对 null/undefined 生效;不能与 && 混用无括号
===	无转换,生产唯一选择;NaN 不自等;对象比引用
== 的坑	0=='0'、''==0、[]==![]、null==0 为 false、null>=0 为 true
判空	x == null 同时匹配 null/undefined,唯一推荐的 == 用法
解构	const [a,b]=arr; const {name}=obj; 可交换变量、取默认值
展开	[...arr] { ...obj } 都是浅拷贝;合并数组/对象最常用
可选链	a?.b?.c 安全取值,与 ?? 搭配是生产标配
求值顺序	严格从左到右(与 C 的 UB 不同)

最小必会清单

- 能说出 5/2、5%2、-5%2、0/0、1/0 的值
- 能说出 + 在字符串与数字混用时变成拼接的规则,并写出规避写法
- 能说出 0xFFFFFFFF|0 的结果并解释 32 位机制
- 能说出 && || 的返回值规则,并写出"默认值"与"条件执行"两种惯用法
- 能列出全部 6 个假值,并解释为什么 '0' 与 [] 是真值
- 能说出 0=='0'、null>=0、[]==![]、null==undefined 四个结果与原因
- 能用 ?? 处理"0 是合法配置"的场景
- 能写出数组/对象解构、展开合并、剩余参数三种代码
- 能说出 ** 的右结合性并写出正确的负底数写法
- 能用可选链与 ?? 重写 5 层嵌套的对象取值

自测题

Q 1. 写出输出:console.log(5/2, 5%2, -5%2, 2**3**2)

✅ 参考答案

2.5、1、-1、512。5/2 是浮点除法;取模符号随被除数;** 右结合:3**2=9 再 2**9=512。

Q 2. 写出输出:console.log(0 && 'a', 1 && 'a', 0 || 'b', 1 || 'b', null ?? 'c', 0 ?? 'c')

✅ 参考答案

0、'a'、'b'、1、'c'、0。&& 返回第一个假值或最后一个值;|| 返回第一个真值或最后一个值;?? 只对 null/undefined 触发。

Q 3. 下面每个表达式的值是什么:0=='0';null>=0;[]==![];null==undefined;'true'==true

✅ 参考答案

true、true、true、true、false。0 与 '0' 比较时字符串转数字;null>=0 中 null 转 0 得 true(而 null==0 是 false);[] 转 "" 后 ![] 转 false,"-==0 两边都转 0;null 与 undefined 互相相等;'true' 转数字是 NaN,与 1 不等。

Q 4. 为什么 const config = {retries: 0}; config.retries || 3 得到 3?正确写法是什么?

✅ 参考答案

0 是假值,|| 返回 3。正确: config.retries ?? 3,?? 只对 null/undefined 触发,保留 0。

Q 5. 用解构与展开把 [1,2] 与 [3,4] 合并并取出第一个元素,再把 {a:1,b:2} 与 {b:9,c:3} 合并。

✅ 参考答案

const [first, ...rest] = [...[1,2], ...[3,4]] (first=1, rest=[2,3,4]); const merged = {...{a:1,b:2}, ...{b:9,c:3}} 得 {a:1,b:9,c:3},后面的 b 覆盖前面的。

Q 6. 判断以下代码的非法点:2 ** -1;console.log(1 ?? 2 || 3);-(2**2)

✅ 参考答案

2 ** -1 合法,得 0.5(指数可为负);1 ?? 2 || 3 抛 SyntaxError,?? 不能与 || 混用不括号; -(2**2) 合法得 -4。

章末实验题:表达式求值器

实验 2:简易表达式求值器

实验目标:实现一个支持加、减、乘、除、取模、幂、括号与一元负号的表达式计算器,用运算符优先级与短路求值实现。

实验要求:

- 任务 1(覆盖:算术运算符、优先级):实现 $+$ $-$ $*$ $/$ $\%$ $**$ 的优先级递归下降解析
- 任务 2(覆盖:括号与一元负号、求值顺序):支持 $($ 与 $)$ 与 $-x$ 一元运算,注意 $**$ 右结合
- 任务 3(覆盖:解构、展开、数组方法):用数组栈管理数字与运算符,或函数递归管理
- 任务 4(覆盖:除法/取模语义、NaN/Infinity): $5/2$ 输出 2.5 ;除零输出 Infinity ;非法输入输出错误提示
- 任务 5(覆盖:逻辑短路、比较、 $??$):空输入用 $??$ 给默认表达式;结果用 $===$ 判断合法性
- 任务 6(覆盖:位运算与自增):附加命令 `bit 7 3` 输出 $7\&3$ 、 $7|3$ 、 7^3 、 $7<<1$ 并演示 32 位溢出($1<<31$ 为负数)

实验环境:Node.js 20+,命令行参数输入表达式: `node calc.js "2 + 3 * 4"`。

第一步 写词法拆分:用正则或手写扫描把字符串拆成 token(数字、运算符、括号)

第二步 写递归下降:`parseExpr`(处理 $+$ $-$)、`parseTerm`(处理 $*$ $/$ $\%$)、`parseFactor`(处理 $**$ 与一元 $-$ 与括号),按优先级逐层调用

第三步 理右结合:幂运算写成右递归或循环内向左收集

第四步 加演示:解析 `bit` 命令并逐行打印位运算结果,直观看到 32 位截断

✅ 参考答案要点

核心:token 数组 + 递归下降。parseExpr→parseTerm→parsePower→parseAtom 四层,幂层用右结合循环。共约 90 行,覆盖本章全部运算符。

```
// calc.js — Node.js 20+ 运行:node calc.js "2 + 3 * 4" 或 node calc.js "bit 7 3"
'use strict';

// 1. 词法分析:拆成 token(覆盖:正则、字符串 API、模板字符串)
function tokenize(src) {
  const tokens = [];
  const re = /\s*([0-9]+(?:\.[0-9]+)?|[\+\-\*\/%()]\s*\s*)/g;
  let m;
  while ((m = re.exec(src)) !== null) {
    tokens.push(m[1]);
  }
  if (tokens.length === 0) return null;
  return tokens;
}

// 2. 递归下降解析器(覆盖:优先级、右结合、一元负号、括号)
class Parser {
  constructor(tokens) {
    this.tokens = tokens;
    this.pos = 0;
  }
  peek() { return this.tokens[this.pos]; }
  next() { return this.tokens[this.pos++]; }

  // 表达式:加/减(最低优先级)
  parseExpr() {
    let left = this.parseTerm();
    while (this.peek() === '+' || this.peek() === '-') {
      const op = this.next();
      const right = this.parseTerm();
      left = { op, left, right };
    }
    return left;
  }

  // 项:乘/除/取模
  parseTerm() {
    let left = this.parsePower();
    while (this.peek() === '*' || this.peek() === '/' || this.peek() === '%') {
      const op = this.next();
      const right = this.parsePower();
      left = { op, left, right };
    }
    return left;
  }

  // 幂:右结合(2**3**2 = 2**(3**2))
  parsePower() {
    const base = this.parseAtom();
    if (this.peek() === '**') {
      this.next();
      const exp = this.parsePower(); // 右递归实现右结合
      return { op: '**', left: base, right: exp };
    }
    return base;
  }
}
```



```

}

// 原子:数字、括号、一元负号
parseAtom() {
  const t = this.peek();
  if (t === '(') {
    this.next();
    const e = this.parseExpr();
    this.next(); // 吃掉 ')'
    return e;
  }
  if (t === '-') {
    this.next();
    return { op: '-', left: { num: 0 }, right: this.parseAtom() };
  }
  this.next();
  return { num: Number(t) };
}
}

```

```

// 3. 求值(覆盖:NaN/Infinity、===、Number.isFinite)
function evaluate(node) {
  if ('num' in node) return node.num;
  const l = evaluate(node.left);
  const r = evaluate(node.right);
  switch (node.op) {
    case '+': return l + r;
    case '-': return l - r;
    case '*': return l * r;
    case '/': return l / r; // 浮点除法,5/2=2.5
    case '%': return l % r; // 符号随被除数
    case '**': return l ** r; // 右结合已在解析阶段处理
    default: return NaN;
  }
}
}

```

```

// 4. 主流程(覆盖:?? 默认值、process.argv、模板字符串、Infinity 提示)
function main() {
  const input = process.argv[2] ?? '1 + 2 * 3';
  if (input.startsWith('bit')) {
    const [_, a, b] = input.split(/\s+/);
    const x = Number(a), y = Number(b);
    console.log(`${x} & ${y} = ${x & y}`);
    console.log(`${x} | ${y} = ${x | y}`);
    console.log(`${x} ^ ${y} = ${x ^ y}`);
    console.log(`1 << 31 = ${1 << 31}(32 位溢出成负数!)`);
    console.log(`(1 << 31) >>> 0 = ${(1 << 31) >>> 0}`);
    return;
  }
  const tokens = tokenize(input);
  if (tokens === null) {
    console.log('错误:空表达式');
    return;
  }
  const ast = new Parser(tokens).parseExpr();
}

```

```
const result = evaluate(ast);
if (Number.isFinite(result)) {
  console.log(`${input} = ${result}`);
} else if (result === Infinity) {
  console.log(`${input} = Infinity(除零)`);
} else {
  console.log(`${input} = NaN(非法运算)`);
}
}

main();
```

设计说明:递归下降天然体现优先级:函数调用层次 = 优先级层次,parsePower 的右递归实现 ** 右结合;一元负号用 0 - x 表示;除零在求值层产生 Infinity,主流程用 Number.isFinite 显式提示,避免静默传播。运行示例: `node calc.js "2 + 3 * 4"` 输出 14, `node calc.js "(2 ** 3) ** 2"` 输出 64, `node calc.js "2 ** 3 ** 2"` 输出 512。

下一章预告:第3章 流程控制 —— 真值判断的 if、严格 === 的 switch、for...of 与 for...in 的选择,以及循环里 var 的经典闭包 bug。